claude-windows / 90c0b6b8-7430-4870-98cb-e487eacfa1d7

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\90c0b6b8-7430-4870-98cb-e487eacfa1d7\subagents\workflows\wf_ad5139d3-ec0\agent-ad06eb99829c69301.jsonl`
- SHA-256: `3cdfbebb87fb708a094fab62bac317efd63661990b9b9a19a59c9b5afbd073a8`
- Source modified: `2026-07-04T01:49:45+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_ad5139d3-ec0`
- session_id: `90c0b6b8-7430-4870-98cb-e487eacfa1d7`

Transcript

1. user (2026-07-04T01:48:32.869Z)

Repo root: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. Read backend/main.py (focus on argparse/CLI and the --server code path) and backend/config/loader.py. Answer precisely, each with file:line evidence:
1. Exact CLI to start the HTTP API server. Confirm whether `python3 -m backend.main --server --host 0.0.0.0 --port 8080` is correct, and list ALL server-mode flags/args accepted.
2. How is config.json located at startup? Confirm the RALLY_APP_HOME env var -> config.json path resolution, and what the default is if RALLY_APP_HOME is unset.
3. Config keys read: confirm indexing.skip_ai_handoff, storage.output_root, storage.input_backend, storage.input_root are honored. List any REQUIRED keys or startup validation (e.g. the loud AI-key startup check from PR #456) that could CRASH the server on boot under a WASB-only (skip_ai_handoff=true, no Gemini key) config. This is critical: I must not deploy a service that crash-loops on boot.
Return structured findings + gaps.

2. assistant (2026-07-04T01:48:35.785Z)

I'll read the two files.

3. user (2026-07-04T01:48:37.138Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1240 of 1625 total (27839 tokens, cap 25000). Call Read with offset=1241 limit=1240 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1       import argparse
2       import json
3       import logging
4       import os
5       import shutil
6       import sys
7       import tempfile
8       import threading
9       import uuid
10      from importlib.metadata import PackageNotFoundError
11      from importlib.metadata import version as _pkg_version
12      from urllib.parse import quote
13
14      import uvicorn
15      from dotenv import load_dotenv
16
17      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the packaged
18      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
```

```
keeps the
19      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
20      # stdlib-only import ? keeps this pre-pydantic line light.
21      from backend.utils.app_home import env_file_to_load, resolve_output_dir
22
23      load_dotenv(env_file_to_load())
24
25      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
26      logging.basicConfig(
27          level=logging.INFO,
28          format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
29          datefmt="%H:%M:%S",
30      )
31      logger = logging.getLogger(__name__)
32      from typing import Any, Dict, List, Optional
33
34      from fastapi import FastAPI, HTTPException, Request
35      from fastapi.middleware.cors import CORSMiddleware
36      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
37      from fastapi.staticfiles import StaticFiles
38      from pydantic import BaseModel
39      from starlette.middleware.trustedhost import TrustedHostMiddleware
40
41      from backend import (
42          billing,  # M8: per-job cost ledger (USD internal / INR display)
43          storage,  # M2: URI-aware input acquisition (local = identity passthrough)
44      )
45      from backend.api import (
46          auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
47      )
48      from backend.config import (
49          AppConfig,
50          StartupCheckError,
51          enforce_startup_checks,
52          write_light_default_config,  # P2b: batteries-included first-run config seeding
53      )
54      from backend.config import (  # new typed config
55          load_config as load_app_config,
56      )
57      from backend.infrastructure.database import Database
58      from backend.pipeline.segmenters.base import segmenter_registry
59      from backend.pipeline.stitcher.compiler import VideoCompiler
60      from backend.pipeline.validators.base import registry
61      from backend.providers import provider_registry
62      from backend.reporting import emit_indexer_report
63      from backend.results import Failure  # standardized error/result contract
64
65      # Server/network bootstrap helpers (CORS/host-allowlist config + the `indexer --app`
launcher)
66      # live in backend.server_runtime (P24 god-module split). Imported HERE, above `app =
FastAPI(...)`,
67      # because the module-level add_middleware(...) below calls
_cors_origins_from_env/_allowed_hosts_from_env
68      # at import time; re-exported so `import backend.main as m; m._resolve_bind_host(...)`
keeps working.
69      from backend.server_runtime import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
70          _allowed_hosts_from_env,
71          _cors_origins_from_env,
72          _find_free_port,
73          _open_browser_when_ready,
74          _port_is_free,
75          _resolve_app_port,
```

```
76          _resolve_bind_host,
77      )
78      from backend.storage.capabilities import (
79          storage_capabilities,  # P3: secret-free "which media can this box use" report
80      )
81      from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
82          ffmpeg_bin,
83          ffprobe_bin,
84      )
85      from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
86      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
87      from backend.utils.paths import (
88          output_base,
89          resolve_under_root,
90          safe_output_filename,
91          validate_input_path,
92      )
93      from backend.utils.redact import (
94          redact_secrets,  # P0b: mask secret-shaped fields in /api/config
95      )
96      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
97
98      app = FastAPI(title="Sports Highlight Indexer API")
99
100
101     def create_app(config: AppConfig, db: Database) -> FastAPI:
102         """Factory: inject shared state via ``app.state`` so route handlers access it
through
103         ``request.app.state`` ? eliminating the Optional module-level globals that forced
104         ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
105
106         Also sets the module-level ``global_config`` / ``db_instance`` for transitional
107         compatibility (helpers and tests still reference them). Route handlers use
108         ``request.app.state`` exclusively."""
109         global global_config, db_instance
110         global_config = config
111         db_instance = db
112         app.state.config = config
113         app.state.db = db
114         return app
115
116
117     # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
118     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
119     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
120     # lock it to its origin; the middleware is fixed at startup.
121     app.add_middleware(
122         CORSMiddleware,
123         allow_origins=_cors_origins_from_env(),
124         allow_credentials=False,
125         allow_methods=["*"],
126         allow_headers=["*"],
127     )
128
129
130     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
131     # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
132     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
133     app.add_middleware(
134         TrustedHostMiddleware,
```

```
        allowed_hosts=_allowed_hosts_from_env(),
        www_redirect=False,
    )


    # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
O1) ---


    # Serves static frontend files directly (now under api/ per approved structure).
    # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
    # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
    # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
    _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
    try:
        os.makedirs(_STATIC_DIR, exist_ok=True)
    except OSError:
        # The dir ships with the package (a no-op create); guard against a read-only install
        # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
        pass
    app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")

    # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
    # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
if it's
    # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
    # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
    # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
    _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
    _UI_DIR = (
        _BUNDLED_UI_DIR
        if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
        else _STATIC_DIR
    )
    _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
    if os.path.isdir(_UI_ASSETS_DIR):
        app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")


    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = os.path.join(_UI_DIR, "index.html")
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"


    def _bundled_ui_version() -> Optional[Dict[str, Any]]:
        """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
        Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
        path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
        except (OSError, ValueError):
```

```
            return None


    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
    # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
    # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
    _instance_lock: Optional[Any] = None

    # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
    # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
    # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
    # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
    # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
    # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
    # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
    from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
        _MAX_DRAIN_WAKE_SEC,
        JobWaiting,
        _arm_wake_timer,
        _drain_due_jobs,
        _get_executor,
        _job_to_request,
        _maybe_record_job_cost,
        _run_claimed_job,
        _schedule_next_drain_if_waiting,
    )

    # The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, Cloud Run
    # dispatch+ingest, compile resolution + worker handler) now lives in
backend.orchestration
    # (P23 god-module split; re-cut of #390). Re-exported here so the names stay addressable
on
    # backend.main ? `import backend.main as m; m._execute_processing(...)`, the job
worker's
    # call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main,
...)
    # all keep working unchanged.
    from backend.orchestration import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
        _cloud_available,
        _CompileError,
        _execute_compile,
        _execute_processing,
        _finalize_reingest,
        _ingest_remote_segments,
        _maybe_dispatch_remote,
        _remote_input_size_bytes,
        _report_config_for_input,
        _resolve_compile,
    )


    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

```

```
244
245     # --- API Models ---
246     class ProcessRequest(BaseModel):
247         video_path: str
248         player_pool: Optional[List[str]] = None
249         max_frames: Optional[int] = None
250         segmenter: Optional[str] = None
251         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
252         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
253         locale: Optional[str] = None
254         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
255         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
256         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
257         compute: Optional[str] = None
258         # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
259         # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
260         force: bool = False
261
262
263     class AnnotationsRequest(BaseModel):
264         # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
265         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
266         video_id: str
267         csv: str
268
269
270     class AssetRequest(BaseModel):
271         # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
272         #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
273         #                   local working copy for the run), OR
274         #   source_uri    ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
275         #                   registered as-is, no copy (D14b).
276         # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
277         # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
278         uploaded_path: Optional[str] = None
279         source_uri: Optional[str] = None
280         medium_uri: str
281         sport: Optional[str] = None
282
283
284     class QueryRequest(BaseModel):
285         video_id: str
286         server: Optional[str] = None
287         receiver: Optional[str] = None
288         duration_min: Optional[float] = None
289         event_type: Optional[str] = None
290
291
292     class CompileRequest(BaseModel):
293         video_id: str
294         segment_ids: List[int]
```

```
295         output_filename: str
296         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
297         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
298         locale: Optional[str] = None
299
300
301     # --- API Endpoints ---
302     @app.get("/api/config")
303     def get_config(request: Request):
304         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
305         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
306         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
307         # by tests/test_redact.py + the /api/config redaction test).
308         cfg = request.app.state.config
309         if cfg is None:
310             return {}
311         return redact_secrets(cfg.model_dump(mode="json"))
312
313
314     @app.get("/api/health")
315     def health(request: Request):
316         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
317         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
318         confirm the engine is reachable and which detector is wired. Never raises."""
319         cfg = request.app.state.config
320         sport = getattr(cfg, "sport", None)
321         indexing = getattr(cfg, "indexing", None)
322         return {
323             "status": "ok",
324             "sport": sport,
325             "default_segmenter": getattr(indexing, "default_segmenter", None),
326         }
327
328
329     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
330     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
331     API_VERSION = 1
332
333
334     def _engine_version() -> str:
335         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
336         try:
337             return _pkg_version("badminton-highlight-indexer")
338         except PackageNotFoundError:
339             return "unknown"
340
341
342     @app.get("/api/version")
343     def api_version():
344         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
345         return {
346             "engine_version": _engine_version(),
347             "api_version": API_VERSION,
348             "ui": _bundled_ui_version(),
349         }
350
```

```
351
352     @app.get("/api/capabilities")
353     def capabilities(request: Request):
354         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
355         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
356         value. Best-effort + never raises (the wizard degrades gracefully)."""
357         cfg = request.app.state.config
358         indexing = getattr(cfg, "indexing", None)
359         deployment = getattr(cfg, "deployment", None)
360         ffmpeg = shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
361         ffprobe = shutil.which(ffprobe_bin())
362         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
363         wasb = getattr(indexing, "wasb", None)
364         weights_path = (
365             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
366         )
367         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
368         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
369         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
370         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
371         nvidia_smi = shutil.which("nvidia-smi")
372         return {
373             "api_version": API_VERSION,
374             "engine_version": _engine_version(),
375             "segmenters": sorted(segmenter_registry.list_available().keys()),
376             "default_segmenter": getattr(indexing, "default_segmenter", None),
377             "ffmpeg": {
378                 "ffmpeg": bool(ffmpeg),
379                 "ffprobe": bool(ffprobe),
380                 "ffmpeg_path": ffmpeg,
381                 "ffprobe_path": ffprobe,
382             },
383             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
384             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
385             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
386             "deployment": {
387                 "profile": getattr(deployment, "profile", "") or "",
388                 "compute_target": getattr(deployment, "compute_target", "") or "",
389                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
390                 # the "run in the cloud" toggle only when true (else it's local-only).
391                 "cloud_available": _cloud_available(cfg),
392             },
393             # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
394             # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
395             "storage": storage_capabilities(
396                 getattr(cfg, "storage", None), local_free_path=output_base(cfg)
397             ),
398         }
399
400
401     def _friendly_video_name(filepath: str) -> str:
402         """A human name for a video from its stored path/URI ? the filename stem
(``?/match_2026.mp4`` ?
403         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
404         Falls back to the whole ``filepath`` if there's no basename."""
```

```
405         base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
406         stem = os.path.splitext(base)[0]
407         return stem or base or str(filepath or "")
408 
409 
410     def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
411         """Pull ``(fps, resolution)`` from the persisted validation results, or ``(None,
None)``. The
412         ``resolution_fps_check`` validator already probed ffprobe and stored
``details={width,height,fps}``
413         on its result (every branch) ? so the videos row carries it with no schema change /
re-probe."""
414         for r in validation_results or []:
415             if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
416                 d = r.get("details") or {}
417                 w, h, fps = d.get("width"), d.get("height"), d.get("fps")
418                 resolution = f"{w}x{h}" if w and h else None
419                 return (fps if isinstance(fps, (int, float)) else None), resolution
420         return None, None
421 
422 
423     def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
424         """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
``id``/``filepath`` (an
425         existing test + any old client rely on them) and adds the fields the SPA library +
the persistent
426         "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
427         ``match_name`` (so an uploaded video shows its filename, not a generic fallback),
``local_path``,
428         ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored
ffprobe
429         validation details) so the library cards can show them. Mirrors the mock engine's
shape so mock and
430         real agree."""
431         fp = row.get("filepath") or ""
432         fps, resolution = _video_dims(row.get("validation_results"))
433         return {
434             **row,
435             "video_id": row.get("id"),
436             "match_name": _friendly_video_name(fp),
437             "local_path": fp,
438             "created_at": row.get("processed_at") or row.get("created_at"),
439             "fps": fps,
440             "resolution": resolution,
441         }
442 
443 
444     @app.get("/api/videos")
445     def list_videos(request: Request, limit: Optional[int] = None):
446         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
447         single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
448         (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
449         names and the persistent "current video" label survives an upload ? see
``_video_summary``."""
450         return {
451             "videos": [
452                 _video_summary(v) for v in request.app.state.db.list_videos(limit=limit)
453             ]
454         }
455 
456 
457     def _reels_dir(cfg) -> str:
```

```
458         return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
459
460
461     _REEL_EXTS = (".mp4", ".mov")
462
463
464     @app.get("/api/reels")
465     def list_reels(request: Request):
466         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
467
468         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
469         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
470         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
471         later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
472         base = _reels_dir(request.app.state.config)
473         reels = []
474         try:
475             entries = os.scandir(base)
476         except OSError:
477             return {"reels": []}
478         with entries:
479             for e in entries:
480                 if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
481                     try:
482                         st = e.stat()
483                     except OSError:
484                         continue
485                     reels.append(
486                         {
487                             "filename": e.name,
488                             "size_bytes": st.st_size,
489                             "modified_at": st.st_mtime,
490                             "download_url": f"/api/reels/{quote(e.name)}",
491                         }
492                     )
493         reels.sort(key=lambda r: r["modified_at"], reverse=True)
494         return {"reels": reels}
495
496
497     @app.get("/api/reels/{filename}")
498     def download_reel(filename: str, request: Request):
499         """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
500         Mirrors /api/highlights' safety (bare-name + containment checks)."""
501         try:
502             name = safe_output_filename(filename)
503         except ValueError as e:
504             raise HTTPException(status_code=400, detail=str(e)) from e
505         base = _reels_dir(request.app.state.config)
506         path = os.path.join(base, name)
507         try:
508             path = resolve_under_root(path, base)
509         except ValueError as e:
510             raise HTTPException(status_code=400, detail=str(e)) from e
511         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
512             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
513         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
514         return FileResponse(path, media_type=media, filename=name)
515
516
```

```
    # --- Annotate-in-UI (P8): serve a video's source + persist finished rally labels
-----------------


    # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to
/api/source
    # don't each kick off a transcode.
    _proxy_inflight: "set[str]" = set()
    _proxy_lock = threading.Lock()
    _source_cache_locks: Dict[str, threading.Lock] = {}
    _source_cache_locks_guard = threading.Lock()


    def _proxies_dir(cfg) -> str:
        return os.path.join(output_base(cfg), "proxies")


    def _source_cache_dir(cfg) -> str:
        return os.path.join(output_base(cfg), "source_cache")


    def _source_cache_lock(video_id: str) -> threading.Lock:
        with _source_cache_locks_guard:
            return _source_cache_locks.setdefault(video_id, threading.Lock())


    def _source_cache_path(video_id: str, input_ref, cfg) -> str:
        ext = os.path.splitext(input_ref.name or "")[1].lower()
        if not ext or "/" in ext or "\\" in ext or len(ext) > 16:
            ext = ".mp4"
        cache_dir = _source_cache_dir(cfg)
        name = safe_output_filename(f"{video_id}{ext}")
        return resolve_under_root(os.path.join(cache_dir, name), cache_dir)


    def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
        """Fetch a remote source once into an output-scoped source/proxy cache."""
        cache_path = _source_cache_path(video_id, input_ref, cfg)
        if os.path.isfile(cache_path):
            return cache_path
        lock = _source_cache_lock(video_id)
        with lock:
            if os.path.isfile(cache_path):
                return cache_path
            cache_dir = os.path.dirname(cache_path)
            os.makedirs(cache_dir, exist_ok=True)
            need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
            require_free_bytes(cache_dir, need, stage="source-cache")
            tmp = os.path.join(
                cache_dir, f".{os.path.basename(cache_path)}.{uuid.uuid4().hex}.tmp"
            )
            try:
                storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
                os.replace(tmp, cache_path)
            finally:
                try:
                    if os.path.exists(tmp):
                        os.remove(tmp)
                except OSError:
                    pass
            return cache_path


    def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
        """Background, best-effort: generate the annotation proxy ONCE so subsequent
```

```
/api/source loads
580         serve the lighter, seekable copy. Deduped; never blocks the request; the source
keeps serving if
581         it fails."""
582         with _proxy_lock:
583             if video_id in _proxy_inflight:
584                 return
585             _proxy_inflight.add(video_id)
586         try:
587             from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
588
589             if not needs_proxy(src_local):
590                 return  # already light enough ? no proxy worth making
591             make_annotation_proxy(
592                 src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
593             )
594         except Exception as e:  # noqa: BLE001 - warming is best-effort
595             logger.warning("proxy: warm failed for %s: %s", video_id, e)
596         finally:
597             with _proxy_lock:
598                 _proxy_inflight.discard(video_id)
599
600
601     @app.get("/api/source/{video_id}")
602     def get_source(video_id: str, request: Request):
603         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
Serves a
604         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
still keyed
605         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
606         background for next time ? non-blocking, so the first load is never delayed. Remote
sources are
607         fetched once into an output-scoped source cache, then served and proxied from that
stable copy."""
608         rec = request.app.state.db.get_video(video_id)
609         if not rec or not rec.get("filepath"):
610             raise HTTPException(status_code=404, detail="Video not found.")
611         cfg = request.app.state.config
612
613         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
614         try:
615             proxy_name = safe_output_filename(f"{video_id}.mp4")
616             proxy_path = resolve_under_root(
617                 os.path.join(_proxies_dir(cfg), proxy_name), _proxies_dir(cfg)
618             )
619         except ValueError:
620             proxy_path = ""
621         if proxy_path and os.path.isfile(proxy_path):
622             return FileResponse(proxy_path, media_type="video/mp4")
623
624         # 2) Otherwise resolve + serve the source.
625         try:
626             input_ref = storage.resolve_input_ref(
627                 rec["filepath"], getattr(cfg, "storage", None)
628             )
629         except (
630             ValueError
631         ) as e:  # unsupported scheme etc. ? a clean client error, never a 500
632             raise HTTPException(status_code=400, detail=str(e)) from e
633         except Exception as e:  # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
634             raise HTTPException(
635                 status_code=502, detail=f"Could not resolve the source video: {e}"
636             ) from e
```

```
637
638         if input_ref.backend == "local":
639             local = input_ref.key
640         else:
641             try:
642                 local = _cached_remote_source(video_id, input_ref, cfg)
643             except HTTPException:
644                 raise
645             except Exception as e:  # noqa: BLE001 - a fetch miss is surfaced (not
swallowed)
646                 raise HTTPException(
647                     status_code=502, detail=f"Could not resolve the source video: {e}"
648                 ) from e
649         if not os.path.isfile(local):
650             raise HTTPException(status_code=404, detail="Source video file not found.")
651
652         # 3) Warm a proxy in the background for next time.
653         threading.Thread(
654             target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
655         ).start()
656
657         media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
658         return FileResponse(local, media_type=media)
659
660
661     @app.post("/api/annotations")
662     def post_annotations(req: AnnotationsRequest, request: Request):
663         """Persist finished rally labels as the annotator CSV (P8), written verbatim to
664         ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5
``video_id`` ? so it joins
665         the video by content hash exactly as the training pipeline expects. Byte-compatible
with the VLC
666         plugin + browser extension. A later step (owner-gated) promotes labels into the
durable corpus.
667
668         Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is
validated as a bare
669         filename so an untrusted value can't escape the labels dir (arbitrary write)."""
670         try:
671             edge_auth.resolve_tenant(request.headers, request.app.state.config)
672         except edge_auth.EdgeAuthError as e:
673             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
674
675         cfg = request.app.state.config
676         labels_dir = os.path.join(output_base(cfg), "labels")
677         try:
678             name = safe_output_filename(f"{req.video_id}.rallies.csv")
679         except ValueError as e:
680             raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
681         os.makedirs(labels_dir, exist_ok=True)
682         path = os.path.join(labels_dir, name)
683         try:
684             path = resolve_under_root(path, labels_dir)
685         except ValueError as e:
686             raise HTTPException(status_code=400, detail=str(e)) from e
687         with open(path, "w", encoding="utf-8", newline="") as f:
688             f.write(req.csv or "")
689         # Count CSV data rows (a line starting with an integer rally_number) for an honest
ack.
690         num_rows = sum(
691             1 for ln in (req.csv or "").splitlines() if ln[:1].isdigit() and "," in ln
692         )
693         logger.info(
694             "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
695         )
```

```
        return {"status": "ok", "num_rows": num_rows, "csv_uri": f"labels/{name}"}


    def _storage_settings(cfg) -> Dict[str, Any]:
        """The storage config as a plain settings dict (NO secrets ? see StorageConfig)."""
        sc = getattr(cfg, "storage", None)
        if sc is None:
            return {}
        return sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)


    def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
        """The durable, MD5-keyed destination inside the chosen medium:
``<medium>/<video_id><ext>``."""
        return f"{medium_uri.rstrip('/')}/{video_id}{ext}"


    def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
        """The LOCAL directory a ``put`` into this medium would write into (so a free-space
BLOCK can be
        enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
blocked).
        ``local`` ? the folder; ``gdrive`` ? the resolved path under the Drive mount."""
        if medium_ref.backend == "local":
            return medium_ref.key or "."
        if medium_ref.backend == "gdrive":
            from backend.storage.capabilities import _gdrive_mount_root

            root = _gdrive_mount_root(storage_settings)
            if not root:
                return None
            rel = medium_ref.key.lstrip("/")
            return os.path.join(root, *rel.split("/")) if rel else root
        return None  # s3 / gcs ? a bucket has no readable local capacity


    @app.post("/api/assets", status_code=201)
    def create_asset(req: AssetRequest, request: Request):
        """Store a video into the user's chosen medium and register it as a library asset
(P4, D14).

        SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
is set ? the
        hosted alpha jails inputs and has no per-user medium credentials, so it stays
upload-to-local.
        Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
local file from
        the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
uploaded file +
        a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
local upload
        stays as the working copy; a ``local`` medium keeps the upload in place (it's
already on this box);
        a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
whole-file
        MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears
in
        ``GET /api/videos``."""
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        cfg = request.app.state.config
        sec = getattr(cfg, "security", None)
```

```
        if getattr(sec, "allowed_video_root", None):
            raise HTTPException(
                status_code=403,
                detail=(
                    "storing into a custom medium is disabled on this hosted deployment "
                    "(security.allowed_video_root is set); upload to local instead."
                ),
            )
        if bool(req.uploaded_path) == bool(req.source_uri):
            raise HTTPException(
                status_code=400,
                detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
            )
        if req.uploaded_path and not req.medium_uri:
            raise HTTPException(
                status_code=400, detail="'medium_uri' is required when uploading a file."
            )

        storage_settings = _storage_settings(cfg)

        # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
D3).
        if req.uploaded_path:
            try:
                validate_input_path(req.uploaded_path, allowed_root=None)
                src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
                raise HTTPException(
                    status_code=404,
                    detail=f"uploaded file not found at '{req.uploaded_path}'",
                )
            local_working = src_ref.key
        else:
            try:
                validate_input_path(req.source_uri, allowed_root=None)
                src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
scratch (P2).
            if src_ref.backend != "local":
                need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
                require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
            try:
                local_working = storage.acquire_local_input(
                    req.source_uri, getattr(cfg, "storage", None)
                )
            except ValueError as e:
                raise HTTPException(status_code=400, detail=str(e)) from e
            except Exception as e:  # noqa: BLE001 ? a fetch miss is a 502, surfaced not
swallowed
                raise HTTPException(
                    status_code=502, detail=f"could not resolve source_uri: {e}"
                ) from e
            if not os.path.isfile(local_working):
                raise HTTPException(
                    status_code=404, detail="source_uri did not resolve to a readable file."
                )

        cleanup_raw = req.source_uri or ""
        try:
            video_id = compute_video_id(local_working)
            ext = os.path.splitext(local_working)[1] or ".mp4"
```

```
811
812             # 2) Decide the durable location + copy only when needed.
813             copied = False
814             if req.source_uri:
815                 stored_uri = (
816                     req.source_uri
817                 )  # already at rest in its medium ? no copy (D14b)
818                 medium_backend = src_ref.backend
819             else:
820                 try:
821                     medium_ref = storage.parse_uri(req.medium_uri)
822                 except ValueError as e:
823                     raise HTTPException(
824                         status_code=400, detail=f"invalid medium_uri: {e}"
825                     ) from e
826                 medium_backend = medium_ref.backend
827                 if medium_ref.backend == "local":
828                     # "This computer" ? the upload already lives here; it IS the durable
copy (no duplicate).
829                     stored_uri = os.path.abspath(local_working)
830                 else:
831                     dest = _asset_dest_uri(req.medium_uri, video_id, ext)
832                     # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud
bucket is warn-only.
833                     local_dir = _medium_local_dir(medium_ref, storage_settings)
834                     if local_dir is not None:
835                         require_free_bytes(
836                             local_dir, os.path.getsize(local_working), stage="store"
837                         )
838                     else:
839                         logger.info(
840                             "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota
warn only).",
841                             medium_ref.backend,
842                         )
843                     try:
844                         stored_uri = storage.put(
845                             local_working, dest, config=storage_settings
846                         )
847                         copied = True
848                     except ValueError as e:
849                         raise HTTPException(status_code=400, detail=str(e)) from e
850                     except Exception as e:  # noqa: BLE001 ? a put failure is surfaced,
never silently dropped
851                         logger.error(
852                             "[ASSETS] put failed video_id=%s dest=%s: %s",
853                             video_id,
854                             dest,
855                             e,
856                             exc_info=True,
857                         )
858                         raise HTTPException(
859                             status_code=502, detail=f"could not store into the medium: {e}"
860                         ) from e
861
862             # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
videos").
863             if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
864                 raise HTTPException(status_code=500, detail="failed to register the asset.")
865             logger.info(
866                 "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
867                 video_id,
868                 medium_backend,
869                 stored_uri,
870                 copied,
```

```
                owner_id,
            )
            return JSONResponse(
                status_code=201,
                content={
                    "video_id": video_id,
                    "medium": medium_backend,
                    "stored_uri": stored_uri,
                    "copied": copied,
                    "sport": req.sport,
                    "status": "stored",
                },
            )
        finally:
            if cleanup_raw:
                storage.cleanup_acquired_local_input(
                    cleanup_raw, local_working, getattr(cfg, "storage", None)
                )


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest, request: Request):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
        # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
        # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        allowed_root = getattr(
            getattr(request.app.state.config, "security", None), "allowed_video_root", None
        )
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(
                req.video_path, getattr(request.app.state.config, "storage", None)
            )
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
```

```
927             raise HTTPException(
928                 status_code=404, detail=f"Video file not found at '{req.video_path}'"
929             )
930         # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
931         # if this box can't hold that copy. Best-effort ? an unstattable remote
(network/SDK) or an
932         # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
933         # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
934         if input_ref.backend != "local":
935             need = _remote_input_size_bytes(
936                 input_ref, getattr(request.app.state.config, "storage", None)
937             )
938             require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
939         if req.segmenter and not segmenter_registry.get(req.segmenter):
940             available = list(segmenter_registry.list_available().keys())
941             raise HTTPException(
942                 status_code=400,
943                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
944             )
945 
946         job_id = uuid.uuid4().hex
947         if not request.app.state.db.create_job(
948             job_id,
949             req.video_path,
950             req.segmenter,
951             req.player_pool,
952             req.max_frames,
953             owner_id=owner_id,
954             locale=req.locale,
955             compute=req.compute,
956             force=req.force,
957         ):
958             raise HTTPException(status_code=500, detail="Failed to persist job record.")
959         _get_executor().submit(
960             _drain_due_jobs, request.app.state.config, request.app.state.db
961         )
962         return JSONResponse(
963             status_code=202,
964             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
965         )
966 
967 
968     @app.get("/api/jobs/{job_id}/cost")
969     def get_job_cost(job_id: str, request: Request):
970         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
971         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
972         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
973         cost = request.app.state.db.get_job_cost(job_id)
974         if cost is None:
975             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
976         fx = request.app.state.config.billing.fx_inr_per_usd
977         usd = cost.get("cost_usd")
978         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
979         cost["fx_inr_per_usd"] = fx
980         return cost
981 
982 
983     @app.get("/api/videos/{video_id}/cost")
984     def get_video_cost(video_id: str, request: Request):
```

```
        """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
        per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
        agg = request.app.state.db.get_video_cost(video_id)
        fx = request.app.state.config.billing.fx_inr_per_usd
        agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
        agg["fx_inr_per_usd"] = fx
        return agg


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str, request: Request):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = request.app.state.db.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }


    # --- Chunked upload (B2, PR-2) ------------------------------------------------
    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
    # sequential-part logic, and abort cleanup ? is unchanged.
    from backend.api import uploads as uploads_api

    app.include_router(uploads_api.router)


    # --- Highlight download (B3, PR-2) --------------------------------------------


    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str, request: Request):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not request.app.state.db.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = (
            getattr(
                getattr(request.app.state.config, "output", None), "output_dir", "output"
            )
            or "output"
```

```
        )
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(
                status_code=404,
                detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
            )
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest, request: Request):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type,
        }
        segments = request.app.state.db.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}


    @app.post("/api/compile", status_code=202)
    def compile_highlights(req: CompileRequest, request: Request):
        """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).

        Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
        can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
        Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
        got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
        fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
`result` carries
        {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
        # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
        try:
            video, _kept, out_name = _resolve_compile(
                request.app.state.db,
                request.app.state.config,
                req.video_id,
                req.segment_ids,
                req.output_filename,
            )
        except _CompileError as e:
            raise HTTPException(status_code=e.status_code, detail=e.detail) from e

        job_id = uuid.uuid4().hex
        # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
the row by id so
        # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
```

```
localized reel.
        if not request.app.state.db.create_compile_job(
            job_id,
            req.video_id,
            video["filepath"],
            req.segment_ids,
            out_name,
            locale=req.locale,
            owner_id=owner_id,
        ):
            raise HTTPException(
                status_code=500, detail="Failed to persist compile job record."
            )
        _get_executor().submit(
            _drain_due_jobs, request.app.state.config, request.app.state.db
        )
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        # Lazy import: cv2 is NOT installed in the LIGHT tier (torch-free default)
        # and this --debug-clip path is CLI-only, not on the served API path.
        import cv2

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

        start_frame = max(0, int((timestamp - 3.0) * fps))
        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))

        # Measure frame-by-frame changes in sample region
        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
        prev_gray = None
        motions = []

        for _ in range(start_frame, end_frame):
            ret, frame = cap.read()
```

```
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (
                    thresh.shape[0] * thresh.shape[1]
                )
                motions.append(motion_ratio)
            prev_gray = gray

        cap.release()

        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(
                f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})"
            )
            if avg_motion > threshold:
                print(
                    "  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play."
                )
            else:
                print(
                    "  Result: [DEAD TIME] Idle movement. Segment classified as dead time."
                )
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)


    def _enforce_startup_or_exit(config) -> None:
        """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
        any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
        the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
        auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
        ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
        the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
        fail-fast wiring + the default-OFF no-op are unit-testable."""
        try:
            enforce_startup_checks(config, logger=logger, include_exposure=True)
        except StartupCheckError as e:
            logger.error("[STARTUP] refusing to start: %s", e)
            sys.exit(1)


```

```
1221    def _maybe_sweep_gemini_orphans(config) -> None:
1222        """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
crashed run.
1223        A no-op unless the configured provider is Gemini AND its API key is present. Never
raises ?
1224        a failed sweep is logged and swallowed. The Gemini client is built with an HTTP
timeout so
1225        a hung connection can't stall startup indefinitely. Extracted for unit tests."""
1226        try:
1227            provider_name = getattr(config, "ai_provider", "")
1228            if provider_name != "gemini":
1229                return
1230            api_key = os.environ.get("GEMINI_API_KEY")
1231            if not api_key:
1232                return
1233            provider = provider_registry.get(
1234                "gemini", api_key=api_key, model_name=config.ai_model
1235            )
1236            deleted = provider.cleanup_orphaned_files()
1237            if deleted:
1238                logger.info(
1239                    "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted
1240                )
```

4. user (2026-07-04T01:48:37.206Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      import os
14      from pathlib import Path
15      from typing import Any
16
17      from pydantic import BaseModel
18
19      from backend.utils.app_home import default_config_path
20
21      from .models import AppConfig
22      from .presets import (
23          COMPUTE_TARGET_FOR_PROFILE,
24          PROFILE_PRESETS,
25          is_known_profile,
26          preset_for,
27          suggest_profile,
28      )
29
30      logger = logging.getLogger(__name__)
31
32      # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
it honours
33      # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
relative) when
34      # the env is unset, exactly the pre-P0a value. (The old module-level
``DEFAULT_CONFIG_PATH`` constant
35      # was dropped: it had no external consumers and a frozen-at-import snapshot would
silently diverge
```

```
    # from a later RALLY_APP_HOME.)


    def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
        """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
        replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
        overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
        path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
        out = dict(base)
        for key, val in override.items():
            if isinstance(val, dict) and isinstance(out.get(key), dict):
                out[key] = _deep_merge(out[key], val)
            else:
                out[key] = val
        return out


    def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
        """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
        is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
        environment never silently selects a profile (and never silently spends on cloud).

        An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
        profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
        asserting a preset that was never applied. A bad value in config.json is left to
surface as
        a hard, typed-Literal error downstream (config-file correctness)."""
        env_profile = os.environ.get("DEPLOYMENT_PROFILE")
        if env_profile and env_profile.strip():
            ep = env_profile.strip()
            if is_known_profile(ep):
                return ep
            logger.warning(
                "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
                "and falling back to config.json deployment.profile.",
                ep,
                ", ".join(PROFILE_PRESETS),
            )
            # fall through to the file's profile (env typo must not suppress a valid file
choice)
        dep = file_data.get("deployment")
        if isinstance(dep, dict):
            prof = dep.get("profile")
            if isinstance(prof, str) and prof.strip():
                return prof.strip()
        return ""


    def _unknown_key_paths(
        data: dict[str, Any], model_cls: type[BaseModel], prefix: str = ""
    ) -> list[str]:
        """Dotted paths in `data` that the typed config will not honor.

        Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
        kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
        typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
        user's intent is lost ? collect every such key so load_config can warn.
        """
```

```
        unknown: list[str] = []
        fields = model_cls.model_fields
        for key, val in data.items():
            if key not in fields:
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if (
                isinstance(val, dict)
                and isinstance(ann, type)
                and issubclass(ann, BaseModel)
            ):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    # Cached model defaults ? Pydantic field defaults are class-level constants
    # that never change at runtime. Initialised lazily on first call to
    # _get_builtin_defaults() so importing this module is still cheap.
    _BUILTIN_DEFAULTS_CACHE: dict[str, Any] | None = None


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        Result is cached ? Pydantic field defaults never change at runtime.
        (The live config returned by load_config() is NOT cached ? it is
        a fresh AppConfig instance every call, reflecting config.json/env.)
        """
        global _BUILTIN_DEFAULTS_CACHE
        if _BUILTIN_DEFAULTS_CACHE is None:
            _BUILTIN_DEFAULTS_CACHE = AppConfig().model_dump(mode="json")
        return _BUILTIN_DEFAULTS_CACHE


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else default_config_path()

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(
                    f"Failed to read {cfg_path}: {exc}. Using defaults + partial data."
                )

        # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
`null`) would
        # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
        # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
```

```
warn, per the
155         # loader's "bad input is non-fatal" contract.
156         if not isinstance(file_data, dict):
157             logger.warning(
158                 "%s is not a JSON object (got %s); ignoring it and using defaults.",
159                 cfg_path,
160                 type(file_data).__name__,
161             )
162             file_data = {}
163 
164         if file_data:
165             unknown = _unknown_key_paths(file_data, AppConfig)
166             if unknown:
167                 logger.warning(
168                     "[CONFIG WARNING] %s contains keys the typed config does not "
169                     "recognize (typo'd or removed knobs ? top-level extras are kept "
170                     "but unread; unknown section keys are silently dropped): %s",
171                     cfg_path,
172                     ", ".join(sorted(unknown)),
173                 )
174 
175         # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
176         # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
177         # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
178         # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
179         defaults = _get_builtin_defaults()
180         profile = _resolve_explicit_profile(file_data)
181 
182         if profile and not is_known_profile(profile):
183             # Only an unrecognised value from config.json reaches here (env typos already
fell back
184             # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
185             # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
186             logger.warning(
187                 "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
188                 profile,
189                 ", ".join(PROFILE_PRESETS),
190             )
191             profile = ""
192 
193         if profile:
194             merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
195             # Record the profile actually applied ? the env may have chosen it over
config.json,
196             # so re-stamp it onto the merged deployment section to keep the record honest.
197             merged.setdefault("deployment", {})
198             if isinstance(merged["deployment"], dict):
199                 merged["deployment"]["profile"] = profile
200                 ct = merged["deployment"].get("compute_target")
201                 canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
202                 if not ct and canonical:
203                     # An active profile with no explicit compute_target (e.g. config.json
blanked it
204                     # to "") gets its canonical target back ? the record never silently lies
about an
205                     # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
206                     merged["deployment"]["compute_target"] = canonical
```

```
                elif ct and canonical and ct != canonical:
                    logger.warning(
                        "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
                        "config.json overrides it to %r ? honoring the explicit override.",
                        profile,
                        canonical,
                        ct,
                    )
                logger.info(
                    "[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
                    profile,
                    merged["deployment"].get("compute_target"),
                )
        else:
            # No profile ? same deep merge so a config.json sub-key overrides its
            # default while NEW defaults added in the model since the file was written
            # survive ? no silent loss of new knobs on upgrade (D4).
            merged = _deep_merge(defaults, file_data)
            suggestion = (
                suggest_profile()
            )  # cheap, env-only (no torch); SUGGESTS only (D15)
            if suggestion:
                logger.debug(
                    "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
                    "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
                    "auto-applies ? D15).",
                    suggestion,
                )

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by scripts/doctor.py diagnostics and the --setup CLI path.
        Overwrites if the file exists.
        """
        cfg_path = Path(path) if path is not None else default_config_path()
        cfg = AppConfig()
        cfg_path.parent.mkdir(parents=True, exist_ok=True)
        cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
        return cfg_path


    # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate O8).
    # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
    # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
    # user gets ? a torch-free, offline, zero-setup experience:
    #   * default_segmenter "motion"  ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
    #   * profile "truly-local"        ? no Gemini handoff, local storage ? never surprise-
bills on
    #                                     Gemini nor returns 0 rallies on WASB-without-a-key
(O8 intent);
    #   * use_gpu false                ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
    # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
    LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
        "sport": "badminton",
```

```
263         "deployment": {"profile": "truly-local"},
264         "indexing": {"default_segmenter": "motion", "use_gpu": False},
265     }
266
267
268     def write_light_default_config(
269         path: str | Path | None = None, *, overwrite: bool = False
270     ) -> tuple[Path, bool]:
271         """Seed the batteries-included LIGHT default config (``LIGHT_DEFAULT_CONFIG``).
272
273         Returns ``(path, written)``. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
274         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
275         overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
276         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
277         """
278         cfg_path = Path(path) if path is not None else default_config_path()
279         if cfg_path.exists() and not overwrite:
280             return cfg_path, False
281         cfg_path.parent.mkdir(parents=True, exist_ok=True)
282         cfg_path.write_text(
283             json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8"
284         )
285         return cfg_path, True
286
287
288     # Backwards-compatible alias used by some older call sites during transition.
289     def get_default_config() -> dict[str, Any]:
290         """Return defaults as a plain dict (for transitional use only)."""
291         return _get_builtin_defaults()
292
```

5. assistant (2026-07-04T01:48:39.117Z)

I have both files. Let me read the rest of main.py to find the `--server` CLI path and argparse.

6. user (2026-07-04T01:48:40.612Z)

```
1241         except Exception as e:
1242             logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1243
1244
1245     def main():
1246         parser = argparse.ArgumentParser(
1247             description="Sports Highlight Indexer Orchestrator"
1248         )
1249         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1250         parser.add_argument(
1251             "--output-dir",
1252             default=None,
1253             help="Directory where processed clips/highlights are saved (overrides
config.json output.output_dir)",
1254         )
1255         parser.add_argument(
1256             "--setup",
1257             action="store_true",
1258             help="Launch dependency checker and setup utility",
1259         )
1260         parser.add_argument(
1261             "--debug-clip",
1262             type=float,
1263             help="Timestamp (in seconds) of a video clip to run detailed diagnostics on",
```

```
        )
        parser.add_argument(
            "--server", action="store_true", help="Run the FastAPI local API server"
        )
        parser.add_argument(
            "--app",
            action="store_true",
            help="Launch the app: start the local server (localhost-only) and open the "
            "bundled UI in your browser. Falls back to a free port if --port is busy.",
        )
        parser.add_argument(
            "--port", type=int, default=8000, help="Port to run the FastAPI server on"
        )
        parser.add_argument(
            "--host",
            default=None,
            help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
            "security.edge_auth_enabled. Expose on a network ONLY behind an auth gate.",
        )
        parser.add_argument(
            "--max-frames",
            type=int,
            help="Limit processing to the first N sub-sampled frames for debugging",
        )

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument(
            "--segmenter",
            choices=available_segmenters,
            help=f"Choose rally detection segmenter. Available: {available_segmenters}",
        )
        parser.add_argument(
            "--trim-start",
            type=float,
            help="Start time in seconds for the debug trim segment feature",
        )
        parser.add_argument(
            "--trim-end",
            type=float,
            help="End time in seconds for the debug trim segment feature",
        )
        parser.add_argument(
            "--trim-output",
            help="Output filename for the debug trimmed segment (saves in output-dir unless
absolute path)",
        )

        args = parser.parse_args()

        if args.setup:
            # Invoke the diagnostics/bootstrap script (renamed from the old root
            # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
            # build system). Resolve its path from this file so `indexer --setup`
            # works regardless of the caller's cwd.
            import subprocess

            doctor_path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                "scripts",
                "doctor.py",
            )
            print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
            subprocess.run([sys.executable, doctor_path])
            return

```

```
1328        if args.trim_start is not None and args.trim_end is not None:
1329            if not args.video_path:
1330                print("[ERROR] Please provide --video-path to extract a segment.")
1331                return
1332
1333            # Determine output path
1334            out_filename = args.trim_output or "trimmed_segment.mp4"
1335            if os.path.isabs(out_filename):
1336                out_path = out_filename
1337                out_dir = os.path.dirname(out_path)
1338                out_name = os.path.basename(out_path)
1339            else:
1340                out_dir = resolve_output_dir(
1341                    args.output_dir or "output"
1342                )  # P0a: app-home aware (CWD-identical when unset)
1343                out_path = os.path.join(out_dir, out_filename)
1344                out_name = out_filename
1345
1346            os.makedirs(out_dir, exist_ok=True)
1347            print(
1348                f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1349            )
1350
1351            # global_config isn't initialized this early; load the typed config so the trim
1352            # clip carries the same configured stitching paddings + watermark as a real
1353            # highlight (watermark default-on).
1354            stitching = load_app_config().stitching
1355            compiler = VideoCompiler(
1356                out_dir,
1357                begin_padding=stitching.begin_padding,
1358                end_padding=stitching.end_padding,
1359                watermark=stitching.watermark,
1360            )
1361            try:
1362                res_path = compiler.compile_highlights(
1363                    args.video_path, [(args.trim_start, args.trim_end)], out_name
1364                )
1365                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1366            except Exception as e:
1367                print(f"[ERROR] Failed to compile trimmed segment: {e}")
1368            return
1369
1370        if args.debug_clip is not None:
1371            if not args.video_path:
1372                print("[ERROR] Please provide --video-path to debug a specific clip.")
1373                return
1374            run_cli_diagnose_clip(args.video_path, args.debug_clip)
1375            return
1376
1377        # P2b (PACKAGING_PLAN.md gate O8): batteries-included first run for the friendly
`--app`
1378        # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
1379        # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
1380        # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
1381        # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
1382        # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1383        # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1384        if args.app:
```

```
1385            seeded_path, seeded = write_light_default_config()
1386            if seeded:
1387                print(
1388                    f"[APP] First run: seeded a truly-local default config at {seeded_path}
"
1389                    f"(detector 'motion' ? offline, no API key or GPU needed). "
1390                    f"Use the in-app setup to switch to Gemini."
1391                )
1392
1393        # Initialize Global Config and DB
1394        global global_config, db_instance
1395        global_config = load_app_config()  # returns typed AppConfig
1396        _enforce_startup_or_exit(
1397            global_config
1398        )  # M5: fail fast on an incoherent ACTIVE profile
1399
1400        # The CLI --output-dir overrides the configured output directory ONLY when
explicitly
1401        # passed. When not passed, the config.json output.output_dir value takes effect.
1402        # It now lives on the typed model (OutputConfig.output_dir), so every consumer ?
including
1403        # /api/compile ? reads the same value instead of a write-only runtime hack.
1404        # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1405        # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1406        if args.output_dir is not None:
1407            global_config.output.output_dir = resolve_output_dir(args.output_dir)
1408        else:
1409            global_config.output.output_dir = resolve_output_dir(
1410                global_config.output.output_dir
1411            )
1412        os.makedirs(global_config.output.output_dir, exist_ok=True)
1413
1414        db_path = os.path.join(
1415            global_config.output.output_dir, global_config.output.db_name
1416        )
1417
1418        # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
1419        # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
1420        # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
1421        # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
1422        global _instance_lock
1423        try:
1424            _instance_lock = acquire_lock(db_path + ".lock")
1425        except OSError as exc:
1426            # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1427            # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1428            # legitimate single instance).
1429            logger.warning(
1430                "[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
1431                "WITHOUT the second-instance guard.",
1432                db_path,
1433                exc,
1434            )
1435            _instance_lock = None
1436        else:
1437            if _instance_lock is None:
1438                print(
```

```
                    f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
                    f"start ? a second instance would corrupt in-flight job state. Stop the
other one, or "
                    f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance."
                )
                sys.exit(1)

        db_instance = Database(db_path)

        # Crash recovery (#16, #329): jobs left queued/running by a previous process are
dead ?
        # the executor is in-process. Compile jobs (idempotent CPU stitch) are RE-QUEUED so
a reel
        # interrupted by a Cloud Run scale-down finishes on the next drain instead of
failing the
        # client; process jobs (GPU, non-idempotent) are failed so pollers see a terminal
state.
        requeued, failed = db_instance.recover_stale_jobs()
        if requeued or failed:
            logger.warning(
                f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-
queued, "
                f"{failed} process failed."
            )

        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
        _maybe_sweep_gemini_orphans(global_config)

        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
        if args.video_path and not args.server and not args.app:
            print(f"[CLI] Processing video file: {args.video_path}")
            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
            storage_cfg = getattr(global_config, "storage", None)
            try:
                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
            except ValueError as e:
                print(f"[FAIL] {e}")
                return
            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return
            local_video = storage.acquire_local_input(args.video_path, storage_cfg)
            try:
                video_id = compute_video_id(local_video)
                was_reingest = db_instance.get_video(video_id) is not None

                # Validate (with-context variant surfaces ffprobe_meta for the report)
                print("[CLI] Validating...")
                val_results, val_context = registry.run_all_with_context(
                    local_video, global_config
                )
                failures = [r for r in val_results if not r.passed]
```

```
1489
1490                # Save video status
1491                status = "failed" if failures else "processed"
1492                db_instance.add_video(
1493                    video_id, args.video_path, status, [r.model_dump() for r in val_results]
1494                )
1495
1496                seg_name = args.segmenter or getattr(
1497                    getattr(global_config, "indexing", None), "default_segmenter", "motion"
1498                )
1499                segmenter_actual = None
1500                segmentation_ran = False
1501                try:
1502                    print("\n" + "=" * 40)
1503                    print("VALIDATION SUMMARY")
1504                    print("=" * 40)
1505                    for r in val_results:
1506                        status_symbol = "[PASS]" if r.passed else "[FAIL]"
1507                        print(f"{status_symbol} {r.validator_name}: {r.message}")
1508                    print("=" * 40 + "\n")
1509
1510                    if failures:
1511                        print("[FAIL] Video failed checks. Indexing halted.")
1512                        return
1513
1514                    # Index
1515                    segmenter_cls = segmenter_registry.get(seg_name)
1516                    if not segmenter_cls:
1517                        print(f"[FAIL] Segmenter '{seg_name}' not found.")
1518                        return
1519
1520                    print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1521                    segmenter_actual = seg_name
1522                    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1523                    segmenter = segmenter_cls(db_instance, global_config)
1524                    result = segmenter.process_video(
1525                        video_id, local_video, max_frames=args.max_frames
1526                    )
1527                    segmentation_ran = True
1528
1529                    if isinstance(result, Failure):
1530                        print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1531                        print("[FAIL] Indexing halted for video.")
1532                        segments = []
1533                    else:
1534                        segments = result.value if hasattr(result, "value") else result
1535                        print(
1536                            f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1537                        )
1538                    # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1539                    _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1540                finally:
1541                    # Always emit the per-video observability report. Never raises.
1542                    paths = emit_indexer_report(
1543                        db=db_instance,
1544                        video_id=video_id,
1545                        # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1546                        # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1547                        # in DB provenance via db.add_video.
1548                        video_path=local_video,
1549                        config=_report_config_for_input(global_config, args.video_path),
```

```
                        val_results=val_results,
                        val_context=val_context,
                        segmenter_requested=args.segmenter,
                        segmenter_actual=segmenter_actual,
                        was_reingest=was_reingest,
                        segmentation_ran=segmentation_ran,
                    )
                    if paths:
                        print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                        print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                        print(f"[REPORT] Machine JSON     : {paths.get('json')}")
            finally:
                storage.cleanup_acquired_local_input(
                    args.video_path, local_video, storage_cfg
                )
            return

        # Start Local Server Mode
        if args.app or args.server or not args.video_path:
            if args.app:
                # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
                # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
                bind_host = "127.0.0.1"
                port = _resolve_app_port(bind_host, args.port)
                if port != args.port:
                    print(f"[APP] Port {args.port} is busy ? using {port} instead.")
                url = f"http://{bind_host}:{port}/"
                # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
                # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
                # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
                print(
                    f"[APP] Starting KhelSutra ? opening {url} in your browser "
                    f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop."
                )
                threading.Thread(
                    target=_open_browser_when_ready,
                    args=(url, url + "api/health"),
                    daemon=True,
                ).start()
            else:
                bind_host = _resolve_bind_host(
                    global_config, args.host
                )  # P0c: config/profile-driven (default 127.0.0.1)
                port = args.port
                print(
                    f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}..."
                )
                if bind_host not in ("127.0.0.1", "localhost", "::1"):
                    print(
                        f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
                        f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
                        f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard
defaults to "
                        f"loopback-only and will 400 forwarded requests otherwise)."
                    )
            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
            # in-process wake-timer for any parked 'waiting' job died with the old process,
```

```
so
1606            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1607            _get_executor().submit(_drain_due_jobs, global_config, db_instance)
1608            # Stamp shared state on the app so route handlers access it via
request.app.state
1609            # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
1610            create_app(global_config, db_instance)
1611            uvicorn.run(app, host=bind_host, port=port)
1612
1613
1614    if __name__ == "__main__":
1615        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1616        # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
1617        # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1618        # `backend.main.db_instance` = None; the async-job drain then crashes silently on
1619        # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
1620        # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
1621        # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
1622        import backend.main as _bm
1623
1624        _bm.main()
1625
```

7. assistant (2026-07-04T01:48:43.445Z)

Now let me look at the startup checks (`enforce_startup_checks`), `_resolve_bind_host`, `default_config_path`, and the config models to answer the config-key and crash-on-boot questions precisely.

8. user (2026-07-04T01:48:46.583Z)

```
backend\config\loader.py:33:# ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals
``Path("config.json")`` (CWD-relative) when
backend\config\loader.py:36:# from a later RALLY_APP_HOME.)
backend\config\startup.py:62:class StartupCheckError(RuntimeError):
backend\config\startup.py:314:def enforce_startup_checks(
backend\infrastructure\database.py:115:                    f"a different data directory
(RALLY_APP_HOME). DB: {self.db_path}"
backend\main.py:17:# PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is
set (the packaged
backend\main.py:384:        "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
backend\main.py:1230:        api_key = os.environ.get("GEMINI_API_KEY")
backend\main.py:1231:        if not api_key:
backend\main.py:1234:            "gemini", api_key=api_key, model_name=config.ai_model
backend\main.py:1380:    # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
backend\main.py:1404:    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME);
with the env
backend\main.py:1441:                f"use a different --output-dir (RALLY_APP_HOME) to run an
independent instance."
backend\eval\gemini_refine.py:17:Run (GEMINI_API_KEY in env):
backend\eval\gemini_refine.py:73:    api_key: str,
backend\eval\gemini_refine.py:86:    provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
backend\eval\gemini_refine.py:138:    api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:139:    if not api_key:
backend\eval\gemini_refine.py:140:        raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:161:                api_key,
backend\eval\gemini_refine.py:230:    api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:231:    if not api_key:
```

```
backend\eval\gemini_refine.py:232:        raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:256:                "gemini", api_key=api_key, model_name=model
backend\api\uploads.py:56:    )  # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME
unset)
backend\pipeline\segmenters\_keyhint.py:10:def api_key_hint(var: str) -> str:
backend\pipeline\segmenters\_keyhint.py:13:        return f'In PowerShell:
$env:{var}="your_api_key_here"'
backend\pipeline\segmenters\_keyhint.py:14:    return f"Set it with: export
{var}=your_api_key_here"
backend\pipeline\segmenters\yolo_hybrid.py:83:        api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\yolo_hybrid.py:84:        api_key = os.environ.get(api_key_env_var)
backend\pipeline\segmenters\yolo_hybrid.py:85:        if not api_key:
backend\pipeline\segmenters\yolo_hybrid.py:86:            from
backend.pipeline.segmenters._keyhint import api_key_hint
backend\pipeline\segmenters\yolo_hybrid.py:89:                f"{api_key_env_var} environment
variable is not set! "
backend\pipeline\segmenters\yolo_hybrid.py:91:                + api_key_hint(api_key_env_var)
backend\pipeline\segmenters\yolo_hybrid.py:97:                provider_name, api_key=api_key,
model_name=model_name
backend\pipeline\segmenters\gemini.py:106:        api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\gemini.py:107:        api_key = os.environ.get(api_key_env_var)
backend\pipeline\segmenters\gemini.py:108:        if not api_key:
backend\pipeline\segmenters\gemini.py:109:            from backend.pipeline.segmenters._keyhint
import api_key_hint
backend\pipeline\segmenters\gemini.py:112:                f"{api_key_env_var} is not set ? the
{provider_name} segmenter cannot run. "
backend\pipeline\segmenters\gemini.py:113:                f"Set your API key. " +
api_key_hint(api_key_env_var)
backend\pipeline\segmenters\gemini.py:120:                provider_name, api_key=api_key,
model_name=model_name
backend\pipeline\segmenters\trajectory_hybrid.py:203:            api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\trajectory_hybrid.py:204:            api_key =
os.environ.get(api_key_env_var)
backend\pipeline\segmenters\trajectory_hybrid.py:205:            if not api_key:
backend\pipeline\segmenters\trajectory_hybrid.py:206:                from
backend.pipeline.segmenters._keyhint import api_key_hint
backend\pipeline\segmenters\trajectory_hybrid.py:209:                    f"{api_key_env_var}
environment variable is not set! "
backend\pipeline\segmenters\trajectory_hybrid.py:211:                    +
api_key_hint(api_key_env_var)
backend\pipeline\segmenters\trajectory_hybrid.py:216:                    provider_name,
api_key=api_key, model_name=model_name
backend\utils\app_home.py:10:  * ``RALLY_APP_HOME`` env set  ? that directory is the app home
(the packaged launcher sets it).
backend\utils\app_home.py:11:  * ``RALLY_APP_HOME`` unset    ? ``Path(".")`` i.e. the CWD,
exactly the pre-P0a behaviour.
backend\utils\app_home.py:15:calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the
resolver opt-in means importing
backend\utils\app_home.py:30:APP_HOME_ENV = "RALLY_APP_HOME"
backend\utils\app_home.py:39:    ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise
``Path(".")`` ? the CWD, byte-identical
backend\utils\app_home.py:53:def default_config_path() -> Path:
backend\utils\app_home.py:56:    With ``RALLY_APP_HOME`` unset this is ``Path("config.json")``
(CWD-relative), unchanged.
backend\utils\app_home.py:62:    """``<app-home>/.env`` ? the dotenv file the server prefers
(e.g. ``GEMINI_API_KEY``)."""
backend\utils\app_home.py:66:def env_file_to_load() -> Path | None:
backend\utils\app_home.py:70:    Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is
set AND that file exists. With the
backend\utils\app_home.py:86:    With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM
(the default ``"output"`` when
backend\utils\app_home.py:105:    ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
```

```
backend\providers\gemini.py:38:    def __init__(self, api_key: str, model_name: str):
backend\providers\gemini.py:39:        super().__init__(api_key, model_name)
backend\providers\gemini.py:47:                api_key=self.api_key,
backend\providers\base.py:6:    def __init__(self, api_key: str, model_name: str):
backend\providers\base.py:7:        self.api_key = api_key
backend\providers\__init__.py:16:    def get(self, name: str, api_key: str, model_name: str) ->
AIVideoProvider:
backend\providers\__init__.py:21:        return self._registry[name](api_key=api_key,
model_name=model_name)
backend\reporting\harvest.py:167:                "api_key_present":
bool(os.environ.get(key_env)) if ai_based else None,
backend\ui\assets\index-Cqr7zJeJ.js:60:[Omitted long matching line]
backend\reporting\spec.yaml:29:      - { path: "stages.ai_handoff.details.api_key_present", op:
eq, value: false }
backend\reporting\spec.yaml:35:      CREDENTIAL, not 'the video has no rallies'. Fix: put
GEMINI_API_KEY=<key> in a
backend\reporting\spec.yaml:69:      - { path: "stages.ai_handoff.details.api_key_present", op:
eq, value: true }
```

9. user (2026-07-04T01:48:46.894Z)

```
1       """Resolved application-data HOME for a packaged, batteries-included install.
2
3       PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4       the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5       ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6       the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
7       "loses" their index. This module introduces ONE resolver every state path roots on.
8
9       Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10        * ``RALLY_APP_HOME`` env set  ? that directory is the app home (the packaged launcher
sets it).
11        * ``RALLY_APP_HOME`` unset    ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13      The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/.local/share``)
14      is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15      calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16      this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17      real user profile during a test run).
18
19      stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20      the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21      """
22
23      from __future__ import annotations
24
25      import os
26      import platform
27      from pathlib import Path
28
29      #: Environment variable the packaged launcher sets to pin the app-data home.
30      APP_HOME_ENV = "RALLY_APP_HOME"
31
32      #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33      APP_DIR_NAME = "Khelsutra"
34
```

```
35
36      def app_home() -> Path:
37          """The resolved application-data home.
38
39          ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40          to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41          """
42          env = os.environ.get(APP_HOME_ENV)
43          if env and env.strip():
44              return Path(env.strip()).expanduser()
45          return Path(".")
46
47
48      def app_path(*parts: str | os.PathLike[str]) -> Path:
49          """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50          return app_home().joinpath(*[str(p) for p in parts])
51
52
53      def default_config_path() -> Path:
54          """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56          With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57          """
58          return app_home() / "config.json"
59
60
61      def default_env_path() -> Path:
62          """``<app-home>/.env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63          return app_home() / ".env"
64
65
66      def env_file_to_load() -> Path | None:
67          """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68          (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69
70          Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71          env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72          prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73          """
74          home = app_home()
75          if (
76              str(home) == "."
77          ):  # env unset (or pinned to CWD) ? historical load_dotenv() search
78              return None
79          candidate = home / ".env"
80          return candidate if candidate.exists() else None
81
82
83      def resolve_output_dir(output_dir: str) -> str:
84          """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
85
86          With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
87          empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
```

```
88          normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
89          ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
90          app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` /
``C:x``) ? passes
91          through untouched.
92          """
93          home = app_home()
94          if str(home) == ".":  # env unset (or pinned to CWD) ? verbatim, today's behaviour
95              return output_dir or "output"
96          if not output_dir:
97              return str(home / "output")
98          if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
99              return output_dir
100         return str(home / output_dir)
101
102
103     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
104         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
105         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
106
107         * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
108         * macOS   ? ``~/Library/Application Support/<app_name>``
109         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
110         """
111         system = platform.system()
112         if system == "Windows":
113             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
114         elif system == "Darwin":
115             base = str(Path.home() / "Library" / "Application Support")
116         else:
117             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
118         return Path(base) / app_name
119
```

10. assistant (2026-07-04T01:48:48.795Z)

Now the critical part ? the startup checks. Let me read `startup.py` fully.

11. user (2026-07-04T01:48:49.996Z)

```
1       """Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
2
3       A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a
coherent
4       runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
5       (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7         * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one
config bug we
8           can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9           bucket can never serve). This is the README §4.3 "loud error before any job".
10        * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
11          falling back to CPU) ? a **warning**, never an error: the probe is best-effort
(torch may live
12          only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the
detector's
13          own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14        * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on
Cloud Run) ?
```

```
15          a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
16          we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18      **Exposure-safety posture (server only, ``include_exposure=True``).** When the engine is
*exposed*
19      behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the
Cloudflare-Access
20      boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the
safety story.
21      A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we
verify the
22      rest of the posture at **boot**: the ``allowed_video_root`` path-jail is set (else an
authenticated
23      tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are
present (else
24      no token can verify ? **error**, lifting the current first-request failure to boot), and
the
25      ``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP
**server** passes
26      ``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant
paths), so it is
27      never "exposed" and stays unaffected.
28
29      **DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the
profile checks are
30      a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False``
(today's default)
31      the exposure checks are a strict **no-op** too. So nothing changes for today's manual-
knob deployments.
32      """
33
34      from __future__ import annotations
35
36      import logging
37      import os
38      from dataclasses import dataclass
39      from typing import Any, Mapping, Optional
40
41      LEVEL_ERROR = "error"
42      LEVEL_WARN = "warn"
43
44      # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
45      _CLOUD_STORAGE = ("gcs", "s3")
46      # STRONG env signals that Google ADC *might* resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /
47      # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
config dir
48      # with no active login), so they would suppress the heads-up falsely. (s3 creds are
intentionally
49      # left to boto3's default chain ? no parallel heads-up, by design.)
50      _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")
51
52
53      @dataclass(frozen=True)
54      class StartupCheck:
55          """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
up)."""
56
57          level: str
58          code: str  # stable, greppable code (e.g. "STORAGE_INCOHERENT")
59          message: str
60
61
```

```
62      class StartupCheckError(RuntimeError):
63          """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
check."""
64
65          def __init__(self, failures: list[StartupCheck]):
66              self.failures = failures
67              super().__init__(
68                  "deployment profile startup check failed: "
69                  + "; ".join(f"[{c.code}] {c.message}" for c in failures)
70              )
71
72
73      def _creds_discoverable(env: Mapping[str, str]) -> bool:
74          """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
False here
75          only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""
76          return any(env.get(k) for k in _GCP_ENV_HINTS)
77
78
79      def _auth_extra_available() -> bool:
80          """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
81          so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically
82          (whether PyJWT is installed in the test env must not change the WARN's presence)."""
83          try:
84              import importlib.util
85
86              return importlib.util.find_spec("jwt") is not None
87          except (
88              Exception
89          ):  # pragma: no cover - find_spec only raises on a broken parent package
90              return False
91
92
93      def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
94          """Exposure-safety posture for an *exposed* HTTP server. Gated on
``security.edge_auth_enabled``
95          (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ?
the "I am
96          exposing this engine behind the CF-Access gate" signal ? the gate alone isn't
enough: verify the
97          rest of the posture at boot so a half-configured exposure fails fast instead of
leaking / 500-ing
98          per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env``
is the
99          caller's environment view (so the host-allowlist check is testable)."""
100         sec = getattr(config, "security", None)
101         if not sec or not getattr(sec, "edge_auth_enabled", False):
102             return []  # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical
to today).
103
104         out: list[StartupCheck] = []
105
106         # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
WITHOUT it,
107         # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
= "any local
108         # file", the single-user default). Fatal for an exposed engine.
109         root = getattr(sec, "allowed_video_root", None)
110         if not root or not str(root).strip():
111             out.append(
112                 StartupCheck(
113                     LEVEL_ERROR,
```

```
                    "EXPOSED_NO_VIDEO_ROOT",
                    "security.edge_auth_enabled is ON (engine exposed) but
security.allowed_video_root is "
                    "unset ? an authenticated tenant could make /api/process read an
arbitrary local file. "
                    "Set allowed_video_root to a jail directory before exposing the
engine.",
                )
            )

        # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
fails on the
        # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
starts
        # "healthy" then 401s every request.
        team = (getattr(sec, "cf_team_domain", "") or "").strip()
        aud = (getattr(sec, "cf_access_aud", "") or "").strip()
        if not team or not aud:
            missing = " + ".join(
                n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud)) if not v
            )
            out.append(
                StartupCheck(
                    LEVEL_ERROR,
                    "EXPOSED_EDGE_AUTH_INCOMPLETE",
                    f"security.edge_auth_enabled is ON but {missing} is not configured ? the
Cf-Access JWT "
                    "cannot be verified (every request would fail). Set
security.cf_team_domain "
                    "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud
(the CF Access "
                    "application AUD tag).",
                )
            )

        # (3) The auth extra (PyJWT[crypto]) is lazy-imported by
auth.verify_cf_access_token. If it's
        # missing, every request 500s at verify time ? a boot WARN beats a per-request 500.
Warning, not
        # fatal: the auth module's own ImportError is the final authority.
        if not _auth_extra_available():
            out.append(
                StartupCheck(
                    LEVEL_WARN,
                    "EXPOSED_AUTH_EXTRA_MISSING",
                    "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ?
Cf-Access "
                    "verification will fail at request time. Install the 'auth' extra (pip
install -e .[auth]).",
                )
            )

        # (4) P0c: an exposed engine binds 0.0.0.0, but the Host-header (DNS-rebinding)
guard defaults to
        # loopback-only when RALLY_ALLOWED_HOSTS is unset (see main._allowed_hosts_from_env)
? it would
        # then 400 every request the edge forwards with its public Host. Boot WARN beats a
silent 400-all.
        if not (env.get("RALLY_ALLOWED_HOSTS", "") or "").strip():
            out.append(
                StartupCheck(
                    LEVEL_WARN,
                    "EXPOSED_HOST_ALLOWLIST_LOOPBACK",
                    "security.edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the
Host-header guard "
```

```
                    "defaults to loopback-only and will 400 forwarded requests. Set
RALLY_ALLOWED_HOSTS to your "
                    "public host(s) (or '*' when the edge gate validates the host).",
                )
            )

        return out


    def run_startup_checks(
        config: Any,
        *,
        cuda_available: Optional[bool] = None,
        env: Optional[Mapping[str, str]] = None,
        include_exposure: bool = False,
    ) -> list[StartupCheck]:
        """Return the startup findings for ``config``.

        ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ? GPU checks
        are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available()``).
        ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the HTTP **server**
        passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both dimensions are
        DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no exposure checks.
        Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns the GPU probe.
        """
        e = os.environ if env is None else env
        checks: list[StartupCheck] = []

        # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box exposure often
        # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
        if include_exposure:
            checks.extend(_exposure_checks(config, e))

        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        if not profile:
            return checks  # no profile ? only the exposure posture (itself off unless edge auth is on).

        storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
        indexing = getattr(config, "indexing", None)
        detector_impl = getattr(indexing, "detector_impl", "auto")
        device = getattr(getattr(indexing, "wasb", None), "device", "cuda")

        if profile == "truly-local":
            # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud backend here is
            # an inconsistent (but not fatal) choice worth surfacing.
            if storage_backend not in ("local", "gdrive"):
                checks.append(
                    StartupCheck(
                        LEVEL_WARN,
                        "STORAGE_UNUSUAL",
                        f"profile 'truly-local' usually uses local/gdrive storage, but storage.backend="
                        f"'{storage_backend}'. Reads will go to a remote backend.",
                    )
                )
            # GPU heads-up (warning only ? the detector healthcheck is the authority; the
```

```
probe is
            # best-effort and may be False even when a subprocess detector env has CUDA).
            if cuda_available is False:
                if device == "cuda":
                    checks.append(
                        StartupCheck(
                            LEVEL_WARN,
                            "CUDA_NOT_VISIBLE",
                            "device='cuda' but no CUDA GPU is visible to this process. If
the box truly "
                            "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
                            "'auto' for the CPU fallback (M4).",
                        )
                    )
                elif device == "auto":
                    checks.append(
                        StartupCheck(
                            LEVEL_WARN,
                            "CPU_FALLBACK",
                            "device='auto' and no CUDA GPU is visible to THIS process ? auto
may resolve "
                            "to CPU (much slower). If torch lives only in the detector
subprocess env this "
                            "can be a false alarm; the runner's DeviceContext/healthcheck is
the authority.",
                        )
                    )

        elif profile == "cloud-serving":
            # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
            if storage_backend not in _CLOUD_STORAGE:
                checks.append(
                    StartupCheck(
                        LEVEL_ERROR,
                        "STORAGE_INCOHERENT",
                        f"profile 'cloud-serving' requires a cloud storage backend "
                        f"({'/'.join(_CLOUD_STORAGE)}), but
storage.backend='{storage_backend}'. "
                        "A cloud job cannot read/write a local-only backend.",
                    )
                )
            elif storage_backend == "gcs" and not _creds_discoverable(e):
                checks.append(
                    StartupCheck(
                        LEVEL_WARN,
                        "CREDS_UNCONFIRMED",
                        "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying
on Application "
                        "Default Credentials. The GCS backend will fail loudly at init if
ADC can't resolve.",
                    )
                )
            # The AI-handoff key: with skip_ai_handoff=False the segmenter demands
            # ${AI_PROVIDER}_API_KEY at rally time and bails to an EMPTY result when it is
            # missing ? a silent 0-rally "success" AFTER the GPU pass is paid for (observed
on
            # the first real cloud-serving litmus, 2026-07-03). The env cannot appear later
in
            # a running container, so this is a zero-false-positive boot-time fatal.
            skip_ai = bool(getattr(indexing, "skip_ai_handoff", False))
            provider = str(getattr(config, "ai_provider", "gemini") or "gemini")
            key_var = f"{provider.upper()}_API_KEY"
            if not skip_ai and not e.get(key_var):
```

```
                checks.append(
                    StartupCheck(
                        LEVEL_ERROR,
                        "AI_HANDOFF_KEY_MISSING",
                        f"profile 'cloud-serving' runs the {provider} handoff "
                        f"(indexing.skip_ai_handoff=False) but {key_var} is not set ? the
segmenter "
                        f"would bail to an empty result after the paid GPU pass. Mount the
key "
                        f"(e.g. a Secret Manager secret) or set
indexing.skip_ai_handoff=true for "
                        "the fully-local candidates-as-rallies mode.",
                    )
                )
            if device == "cpu":
                checks.append(
                    StartupCheck(
                        LEVEL_WARN,
                        "CLOUD_CPU_DEVICE",
                        "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
                        "detection will not use the GPU.",
                    )
                )
            elif cuda_available is False and device in ("cuda", "auto"):
                checks.append(
                    StartupCheck(
                        LEVEL_WARN,
                        "CLOUD_CUDA_NOT_VISIBLE",
                        "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
                    )
                )

        elif profile == "cpu-mock":
            # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
            if detector_impl != "stub":
                checks.append(
                    StartupCheck(
                        LEVEL_WARN,
                        "MOCK_NONSTUB_DETECTOR",
                        f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
                        f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
                    )
                )

        return checks


    def enforce_startup_checks(
        config: Any,
        *,
        cuda_available: Optional[bool] = None,
        env: Optional[Mapping[str, str]] = None,
        raise_on_error: bool = True,
        include_exposure: bool = False,
        logger: Optional[logging.Logger] = None,
    ) -> list[StartupCheck]:
        """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
        ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.

        ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the
full findings
```

```
327         list (so a caller can inspect/surface them). A no-op with no logging and no raise
when both
328         dimensions are off (``run_startup_checks`` returns ``[]``)."""
329         log = logger or logging.getLogger(__name__)
330         checks = run_startup_checks(
331             config,
332             cuda_available=cuda_available,
333             env=env,
334             include_exposure=include_exposure,
335         )
336         for c in checks:
337             if c.level == LEVEL_ERROR:
338                 log.error("[STARTUP] %s: %s", c.code, c.message)
339             else:
340                 log.warning("[STARTUP] %s: %s", c.code, c.message)
341         errors = [c for c in checks if c.level == LEVEL_ERROR]
342         if errors and raise_on_error:
343             raise StartupCheckError(errors)
344         return checks
345
```

12. assistant (2026-07-04T01:48:54.922Z)

I now have everything needed. Let me confirm the storage config keys mentioned (`storage.output_root`, `storage.input_backend`, `storage.input_root`) against the actual model, since main.py reads `output.output_dir` not `storage.output_root`.

13. user (2026-07-04T01:48:56.925Z)

```
180:class IndexingConfig(BaseModel):
267:    skip_ai_handoff: bool = False
302:class OutputConfig(BaseModel):
307:    output_dir: str = "output"
391:class StorageConfig(BaseModel):
399:    backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
401:    output_root: str = ""
402:    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
405:    # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
406:    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
410:    input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
411:    input_root: str = ""
418:    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
441:    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.
495:class AppConfig(BaseModel):
```

14. assistant (2026-07-04T01:49:01.618Z)

I have all the evidence. Let me read the StorageConfig block and the AppConfig fields briefly to confirm defaults and precedence.

15. user (2026-07-04T01:49:02.573Z)

```
391     class StorageConfig(BaseModel):
392         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
393         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
394         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
395         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
396         **secrets are never stored here**, only endpoints / regions / file paths / source-
```

```
selectors
397         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
398
399         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
400         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
401         output_root: str = ""
402         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
403         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
404         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
405         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
406         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
407         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
408         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
409         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
410         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
411         input_root: str = ""
412         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
413         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
414         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
415         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
416         # the legacy flat layout is used unchanged.
417         single_folder_per_video: bool = False
418         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
419         workspace_root: str = ""
420         # Cloud namespace prefix under the root so per-video folders sit tidily beside
421         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
422         workspace_prefix: str = "workspaces"
423         local: dict[str, Any] = Field(default_factory=dict)
424         s3: dict[str, Any] = Field(default_factory=dict)
425         gcs: dict[str, Any] = Field(default_factory=dict)
426         gdrive: dict[str, Any] = Field(default_factory=dict)
427         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
428         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
429         gdrive_api: dict[str, Any] = Field(default_factory=dict)
430
431
432     class DeploymentConfig(BaseModel):
433         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
434         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
435
436         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
437         behaves exactly as before this section existed. A profile is opt-in (``config.json``
438         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
439         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
440         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
441         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
```

```
loader.
442
443         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
444
445         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
446         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
447         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
448         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
449         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
450
```

16. assistant (2026-07-04T01:49:04.482Z)

I have complete evidence. Providing structured findings.

17. user (2026-07-04T01:49:44.931Z)

Structured output provided successfully